

LibeRator reference manual

Adaptive Therapeutic Optimisation and Recommendation

Version 0.2.3

Generated 2026-07-22

30 help topics

Contents

- [lator_assess](#)
- [lator_covariate_at](#)
- [lator_endpoint](#)
- [lator_endpoint_aed](#)
- [lator_endpoint_atg](#)
- [lator_endpoint_beta_lactam](#)
- [lator_endpoint_evaluate](#)
- [lator_endpoint_library](#)
- [lator_endpoint_register](#)
- [lator_endpoint_validate](#)
- [lator_example_aed](#)
- [lator_gui](#)
- [lator_job](#)
- [lator_model_from_library](#)
- [lator_model_register](#)
- [lator_patient_add_event](#)
- [lator_patient_add_events](#)
- [lator_patient_get](#)
- [lator_patient_list](#)
- [lator_patient_new](#)
- [lator_patient_save](#)
- [lator_regimen_candidates](#)
- [lator_regimen_optimise](#)
- [lator_regimen_predict](#)
- [lator_worker_task](#)
- [lator_workspace](#)
- [lator_workspace_audit](#)
- [liberator_workbench](#)
- [liberatorWorkbenchOutput](#)
- [renderLiberatorWorkbench](#)

Longitudinal Bayesian patient assessment

Description

Each call creates an immutable assessment tied to hashes of its patient evidence, model, endpoint, covariate policy, and cutoff. 'dynamic' mode fits occasion-specific ETA states under a Gaussian random-walk prior while the compartment state remains continuous across boundaries.

Usage

```
lator_assess(  
  patient,  
  model,  
  endpoint,  
  analyte = endpoint$drug,  
  cutoff = Inf,  
  mode = c("static", "dynamic"),  
  state_times = NULL,  
  covariate_policies = list(),  
  process_scale = 0.1,  
  process_covariance = NULL,  
  workspace = NULL,  
  actor = "local-session",  
  ...  
)
```

Arguments

patient	Longitudinal patient record.
model	LibeRation population model.
endpoint	Versioned therapeutic endpoint.
analyte	Event drug/analyte name; defaults to 'endpoint\$drug'.

cutoff	Patient-timeline hour through which evidence is visible.
mode	"static" or "dynamic" latent patient parameters.
state_times	Optional dynamic-state boundaries; otherwise explicit 'state_boundary' events are used.
covariate_policies	Named per-covariate policy lists passed to [lator_covariate_at()].
process_scale	Random-walk innovation covariance as a multiple of OMEGA.
process_covariance	Optional explicit innovation covariance.
workspace	Optional workspace. When supplied, the immutable assessment is appended to and saves the patient record.
actor	Audit actor used when saving.
...	Additional arguments to [LibeRation::nm_individual_fit()].

Resolve a longitudinal covariate at specified times

Description

Missing observations are never silently converted into population values. Carry-forward, interpolation, nearest-neighbour, and fallback behaviour are explicit policies and every returned value records its provenance and age.

Usage

```
lator_covariate_at(  
  patient,  
  name,  
  times,  
  method = c("locf", "linear", "nearest", "none"),  
  max_age = Inf,  
  max_gap = Inf,  
  fallback = NULL,  
  fallback_unit = "",  
  uncertainty_sd = NA_real_,  
  cutoff = Inf  
)
```

Arguments

patient	A 'lator_patient'.
name	Covariate event name.
times	Numeric timeline hours.
method	One of ""locf"", ""linear"", ""nearest"", or ""none"".
max_age	Maximum permitted age in hours for carried or nearest values.

max_gap	Maximum interval in hours spanning a linear interpolation.
fallback	Optional explicit fallback value. It is labelled 'fallback', never as an observation.
fallback_unit	Unit for 'fallback' if no observed unit is available.
uncertainty_sd	Optional standard deviation attached to fallback values. Event-specific 'metadata\$uncertainty_sd' takes precedence for observed values.
cutoff	Ignore evidence recorded after this patient-timeline time.

Value

A provenance-rich data frame with one row per requested time.

Define a versioned therapeutic endpoint

Description

Endpoint definitions are evidence-bearing data, separate from estimation and optimisation code. 'draft' endpoints are suitable for exploration; a 'qualified' status is an explicit governance assertion by the user, not a certification made by LibeRator.

Usage

```
lator_endpoint(  
  id,  
  name,  
  drug,  
  kind,  
  metric,  
  unit = "",  
  rules = list(),  
  source = "",  
  status = c("draft", "reviewed", "qualified"),  
  version = "1.0.0",  
  metadata = list()  
)
```

Arguments

id	Stable endpoint identifier.
----	-----------------------------

name	Display name.
------	---------------

drug	Drug or intervention name.
------	----------------------------

kind	Endpoint kind.
------	----------------

metric	Metric evaluated against predicted trajectories.
--------	--

unit	Endpoint unit.
rules	Named rule list interpreted by the endpoint evaluator.
source	Citation or institutional-policy provenance.
status	"draft", "reviewed", or "qualified".
version	Semantic version string.
metadata	Additional non-executable metadata.

Value

A serializable 'lator_endpoint'.

AED therapeutic-range endpoint

Description

The optimisation target is the midpoint of the supplied range. No drug ranges are hard-coded because ranges depend on assay, matrix, indication, population, sampling time, and institutional policy.

Usage

```
lator_endpoint_aed(  
  drug,  
  lower,  
  upper,  
  unit,  
  source,  
  status = c("draft", "reviewed", "qualified"),  
  metric = "last_interval_average"  
)
```

Arguments

drug	Drug name.
lower, upper	Therapeutic-range bounds.
unit	Concentration unit.
source	Evidence or policy provenance.
status	Governance status.
metric	Prediction metric; by default average concentration over the last dose interval.

ATG pre-transplant target endpoint

Description

ATG pre-transplant target endpoint

Usage

```
lator_endpoint_atg(  
  drug,  
  targets,  
  unit,  
  source,  
  anchor = "transplantation",  
  status = c("draft", "reviewed", "qualified")  
)
```

Arguments

drug	ATG product/name.
targets	Data frame with 'window_start', 'window_end', 'lower', and 'upper', expressed relative to the anchor in hours.
unit	Concentration or exposure unit.
source	Evidence or policy provenance.
anchor	Patient procedure-event name used as time zero.
status	Governance status.

Beta-lactam fraction-of-time-above-MIC endpoint

Description

Beta-lactam fraction-of-time-above-MIC endpoint

Usage

```
lator_endpoint_beta_lactam(  
  drug,  
  target_fraction = 0.4,  
  mic_variable = "MIC",  
  threshold_multiplier = 1,  
  free_fraction = 1,  
  source,  
  status = c("draft", "reviewed", "qualified")  
)
```

Arguments

drug	Drug name.
target_fraction	Required fraction of an interval above threshold.
mic_variable	Covariate/event name containing MIC.
threshold_multiplier	Threshold as a multiple of MIC.
free_fraction	Fraction unbound; use one if predictions are already free concentrations.
source	Evidence or policy provenance.

status

Governance status.

Evaluate predictions against a therapeutic endpoint

Description

Evaluate predictions against a therapeutic endpoint

Usage

```
lator_endpoint_evaluate(endpoint, predictions, patient = NULL, interval = NULL)
```

Arguments

endpoint	Endpoint definition.
predictions	Prediction data frame. Optional 'SIM' identifies uncertainty replicates.
patient	Patient timeline, required when an endpoint obtains MIC or an event anchor from it.
interval	Optional two-element evaluation interval.

Value

Per-replicate metrics and an aggregate target-attainment summary.

List built-in templates and registered endpoint definitions

Description

List built-in templates and registered endpoint definitions

Usage

```
lator_endpoint_library(workspace = NULL)
```

Arguments

workspace

Optional unlocked workspace.

Register an encrypted endpoint definition

Description

Register an encrypted endpoint definition

Usage

```
lator_endpoint_register(workspace, endpoint, actor = "local-session")
```

Arguments

workspace	Unlocked workspace.
endpoint	Endpoint to register.
actor	Audit actor.

Validate an endpoint definition

Description

Validate an endpoint definition

Usage

```
lator_endpoint_validate(endpoint)
```

Arguments

endpoint	Endpoint object.
----------	------------------

Value

The validated endpoint.

Synthetic AED teaching case

Description

Creates a small, non-clinical longitudinal case containing an ADVAN1 model, intermittent weight measurements, repeat dosing, TDM observations, a latent state boundary, and an illustrative therapeutic-range endpoint. It is meant for software teaching and tests; its target range is not a real dosing rule.

Usage

```
lator_example_aed()
```

Value

Named list with 'model', 'patient', and 'endpoint'.

Launch the LibeRator longitudinal dosing workbench

Description

By default the server binds only to loopback. Exposing it to another host is deliberately blocked unless 'allow_remote = TRUE'; production deployment additionally requires institutional authentication, TLS, authorization, backup, monitoring, validation, and clinical governance outside this research package.

Usage

```
lator_gui(  
  workspace = NULL,  
  path = NULL,  
  passphrase = NULL,  
  key = NULL,  
  models = NULL,  
  endpoints = NULL,  
  host = "127.0.0.1",  
  port = NULL,  
  launch.browser = TRUE,  
  allow_remote = FALSE,  
  session_workspace = FALSE,  
  teaching_example = FALSE  
)
```

Arguments

workspace	Optional unlocked workspace.
path	Workspace path when 'workspace' is not supplied.
passphrase	Optional passphrase. If omitted, an unlock screen is shown.
key	Optional managed 32-byte key.

<code>models</code>	Session models in addition to encrypted registrations.
<code>endpoints</code>	Session endpoints in addition to encrypted registrations.
<code>host, port, launch.browser</code>	Passed to <code>[shiny::runApp()]</code> .
<code>allow_remote</code>	Explicitly permit a non-loopback bind for governed test deployments.
<code>session_workspace</code>	Create a separate ephemeral encrypted workspace for every browser session. This is intended for hosted research demonstrations and prevents application users from sharing a workspace directory.
<code>teaching_example</code>	Seed an otherwise empty workspace with the synthetic AED teaching patient, model, and endpoint. Intended for demonstrations; the example is explicitly non-clinical.

Value

Invisibly, the Shiny app.

Create a deidentified LibeRties individualisation or regimen job

Description

The caller supplies model-ready event data rather than a LibeRator patient record. Workspace keys, patient catalogues, audit records, and direct identifiers are therefore never included in a queue payload.

Usage

```
lator_job(  
  type = c("individualise", "regimen"),  
  model,  
  data,  
  arguments = list(),  
  label = NULL  
)
```

Arguments

type	"individualise" or "regimen".
model	Serializable LibeRation model.
data	Deidentified model-ready data. Regimen jobs require a 'CANDIDATE' column.
arguments	Named worker arguments.
label	Job label.

Value

A LibeRties job ready for a local or remote queue.

Import a LibeRary catalogue model

Description

Unvalidated catalogue entries are rejected by default. Enabling them is an explicit research override retained in model provenance.

Usage

```
lator_model_from_liberary(library_id, root = NULL, allow_unvalidated = FALSE)
```

Arguments

library_id	LibeRary catalogue id.
root	LibeRary catalogue root.
allow_unvalidated	Permit an entry whose status is not 'validated'.

Value

A compiled, serializable LibeRation 'nm_model' with catalogue provenance.

Register an encrypted LibeRation model for individualisation

Description

Register an encrypted LibeRation model for individualisation

Usage

```
lator_model_register(  
  workspace,  
  model,  
  id = NULL,  
  name = NULL,  
  qualification = list(status = "research"),  
  endpoint_ids = character(),  
  provenance = list(),  
  actor = "local-session"  
)
```

Arguments

workspace	Unlocked workspace.
model	A LibeRation 'nm_model'.
id	Stable local model id.
name	Display name.
qualification	Named governance and validation metadata.
endpoint_ids	Compatible endpoint identifiers.

provenance

Evidence and import provenance.

actor

Audit actor.

Add an immutable event to a patient timeline

Description

Corrections are represented by a new event whose 'supersedes' field points to the earlier event. Existing evidence is never silently overwritten.

Usage

```
lator_patient_add_event(  
  patient,  
  type,  
  time,  
  name = "",  
  value = NA,  
  unit = "",  
  source = "manual",  
  missing_reason = "",  
  occasion = NA_integer_,  
  supersedes = "",  
  metadata = list()  
)
```

Arguments

patient	Patient record.
type	Event type.
time	Numeric hours on the patient's study timeline, or a POSIX time.
name	Clinical/model variable name.
value	Scalar value.

unit	Unit string.
source	Provenance label.
missing_reason	Reason a scheduled value is unavailable.
occasion	Optional dynamic-parameter epoch.
supersedes	Earlier event id corrected by this event.
metadata	Additional typed event fields such as route, duration, CMT, LLOQ, assay, pathogen, MIC method, or adherence certainty.

Value

Updated patient record.

Add several event records

Description

Add several event records

Usage

```
lator_patient_add_events(patient, events)
```

Arguments

patient	Patient record.
---------	-----------------

events	Data frame or list of named event argument lists.
--------	---

Value

Updated patient record.

Load a patient

Description

Load a patient

Usage

```
lator_patient_get(workspace, patient_id)
```

Arguments

workspace	Unlocked workspace.
-----------	---------------------

patient_id	Pseudonym.
------------	------------

List pseudonymous patient records

Description

List pseudonymous patient records

Usage

```
lator_patient_list(workspace)
```

Arguments

workspace	Unlocked workspace.
-----------	---------------------

Value

Patient catalogue data frame.

Create a pseudonymous longitudinal patient record

Description

Direct identifiers such as name, address, date of birth, hospital number, email, or telephone number are intentionally absent. Linkage to an identifiable clinical record belongs in the controlling institution's separately governed system.

Usage

```
lator_patient_new(patient_id, study_id = "", label = "", metadata =  
list())
```

Arguments

patient_id	Pseudonym assigned outside LibeRator.
study_id	Optional research-study identifier.
label	Non-identifying display label.
metadata	Non-identifying study metadata.

Value

A 'lator_patient'.

Persist a patient with optimistic revision checking

Description

Persist a patient with optimistic revision checking

Usage

```
lator_patient_save(  
  workspace,  
  patient,  
  expected_revision = patient$revision,  
  actor = "local-session"  
)
```

Arguments

workspace	Unlocked workspace.
patient	Patient record.
expected_revision	Expected stored revision; defaults to the record's current revision. Prevents two sessions overwriting one another.
actor	Audit actor.

Value

Saved patient with incremented revision.

Construct a clinically feasible regimen grid

Description

Construct a clinically feasible regimen grid

Usage

```
lator_regimen_candidates(  
  amounts,  
  intervals,  
  routes = "oral",  
  infusion_durations = 0,  
  loading_doses = NA_real_,  
  horizon = 168  
)
```

Arguments

amounts	Dose amounts.
intervals	Dose intervals in hours.
routes	Route labels.
infusion_durations	Infusion durations in hours; zero is an instantaneous input to the model's dose compartment.
loading_doses	Optional loading amounts; 'NA' means no separate load.
horizon	Evaluation horizon in hours.

Value

Candidate-regimen data frame.

Compare candidate regimens under posterior uncertainty

Description

The complete dosing history is replayed for every posterior draw so that accumulated compartment amounts are retained. Parameter uncertainty is evaluated in one batched C++ simulation call per candidate.

Usage

```
lator_regimen_optimise(  
  assessment,  
  patient,  
  candidates,  
  endpoint = assessment$endpoint,  
  nsim = 200L,  
  grid_step = 0.25,  
  start_time = NULL,  
  residual = FALSE,  
  max_daily_dose = Inf,  
  max_single_dose = Inf,  
  dose_burden_weight = 0,  
  seed = NULL,  
  n_cores = 1L  
)
```

Arguments

assessment	A completed 'lator_assessment'.
patient	The corresponding patient timeline.
candidates	Candidate data frame from [lator_regimen_candidates()].
endpoint	Endpoint to optimise; defaults to the assessment endpoint.

<code>nsim</code>	Number of posterior draws.
<code>grid_step</code>	Prediction-grid spacing in hours.
<code>start_time</code>	First candidate dose time; defaults just after the latest historical event visible to the assessment.
<code>residual</code>	Include residual observation variability in attainment probabilities.
<code>max_daily_dose</code>	Optional hard feasibility constraint.
<code>max_single_dose</code>	Optional hard feasibility constraint.
<code>dose_burden_weight</code>	Small optional tie-breaker applied to daily dose.
<code>seed</code>	Reproducible seed.
<code>n_cores</code>	Simulation cores used within LibeRation.

Value

A ranked result with summaries and per-candidate trajectories.

Create an explicit future prediction for a selected regimen

Description

Regimen comparison already simulates every feasible candidate under posterior uncertainty. This function promotes one candidate trajectory into a separate, auditable forecast artifact without repeating the expensive simulation or treating the highest-ranked row as an automatic dosing decision.

Usage

```
lator_regimen_predict(  
  comparison,  
  candidate_id,  
  probs = c(0.05, 0.5, 0.95)  
)
```

Arguments

comparison	Result from <code>lator_regimen_optimise()</code> .
------------	---

candidate_id	Candidate deliberately selected by the user.
--------------	--

probs	Three ordered probabilities used for the lower interval, median, and upper interval.
-------	--

Value

A `lator_future_prediction` containing pointwise uncertainty bands, the selected regimen, linked assessment and endpoint evaluation.

Execute a typed LibeRator worker task

Description

This is the narrow entry point called by LibeRties workers. It accepts no workspace object or encryption key.

Usage

```
lator_worker_task(type, model, data, arguments = list())
```

Arguments

type	Worker task.
model	Serializable model.
data	Model-ready data.
arguments	Named controls.

Open an encrypted LibeRator workspace

Description

Patient records, the patient catalogue, endpoints, model registrations and the audit chain are encrypted and authenticated with libsodium. The key is derived with Argon2 and is never written to the workspace. Direct patient identifiers are deliberately outside the schema; use a pseudonym generated by the controlling study or institution.

Usage

```
lator_workspace(path = NULL, passphrase = NULL, key = NULL, create = TRUE)
```

Arguments

path	Workspace directory.
passphrase	Session passphrase of at least 12 characters.
key	Optional 32-byte raw key for managed deployments.
create	Create a missing workspace.

Value

A key-bearing 'lator_workspace'. Do not serialize or transmit it.

Read and verify the encrypted workspace audit chain

Description

Read and verify the encrypted workspace audit chain

Usage

```
lator_workspace_audit(workspace)
```

Arguments

workspace	Unlocked workspace.
-----------	---------------------

Value

Audit events with a 'valid' attribute.

LibeRator React workbench widget

Description

LibeRator React workbench widget

Usage

```
liberator_workbench(  
  payload,  
  input_id = "liberator_workbench",  
  width = NULL,  
  height = "100vh",  
  elementId = NULL  
)
```

Arguments

payload	Workbench payload generated by the Shiny application.
input_id	Shiny event prefix.
width, height	Widget dimensions.
elementId	Optional widget element id.

Shiny output for the LibeRator workbench

Description

Shiny output for the LibeRator workbench

Usage

```
liberatorWorkbenchOutput(outputId, width = "100%", height = "100vh")
```

Arguments

outputId	Output id.
width, height	CSS dimensions.

Render a LibeRator workbench

Description

Render a LibeRator workbench

Usage

```
renderLiberatorWorkbench(expr, env = parent.frame(), quoted = FALSE)
```

Arguments

expr	Widget expression.
env	Evaluation environment.
quoted	Whether expression is quoted.
